PRODUCT OPPORTUNITY / 15 OCTOBER 2026

The missing scenarios behind green tests

Market study and product strategy for a simple, open-source tool that works with Claude Code and Codex. Prepared for Omkar Ghugarkar.

Build a change-specific scenario coverage tool. Make the gap visible, turn accepted gaps into normal tests, and attach fresh execution evidence.

Market reality	Implication for your product
Scenario testing is established. BDD, model-based testing and AI platforms already address parts of it.	Use existing test runners and specification formats. Adoption depends on less work for the developer.
Native agents can already plan, run and repair tests.	Easy setup and subscription integration are necessary, but do not establish an advantage.
Coverage can look complete while business behavior remains unchecked.	The product should explain its denominator, assertion links, stale results and unknowns.

Scope: 53 product/framework families, chiefly for web/API business-scenario testing, plus adjacent tools and native-agent substitutes. This is a representative market map, not an exhaustive directory. Features are documented vendor claims; products were not benchmarked hands-on. Prices are public snapshots, not negotiated quotes. Recommendations, prices for your product, timelines and success thresholds are hypotheses.

Primary-source references are clickable throughout. Some dynamic sites could only be checked through indexed official content; those limitations are marked. Research cutoff: 15 October 2026.

What scenario testing actually measures

Unit, integration and end-to-end describe the scope of a test. A scenario describes the behavior under particular conditions. You can test a scenario at any of those levels. A business rule is often cheapest to verify with a focused unit or API test; a small number of browser journeys can verify the complete experience.

Coverage measure	Question answered	What it leaves uncertain
Line / branch coverage	Did tests execute these statements or decisions?	Whether the expected business result was checked.
Requirement coverage	Which agreed requirements have tests and results?	Whether the requirements themselves are complete.
Scenario coverage	Which defined actors, states, events and outcomes were exercised?	Behaviors missing from the scenario inventory.
State / transition coverage	Which modeled states and transitions were visited?	Long sequences and missing model elements.
Interaction coverage	Which valid t-way input/configuration combinations were exercised?	Higher-order and temporal interactions beyond the chosen model.
Mutation score	Do tests detect selected changes to the implementation?	Whether business intent and the mutation set are sufficient.

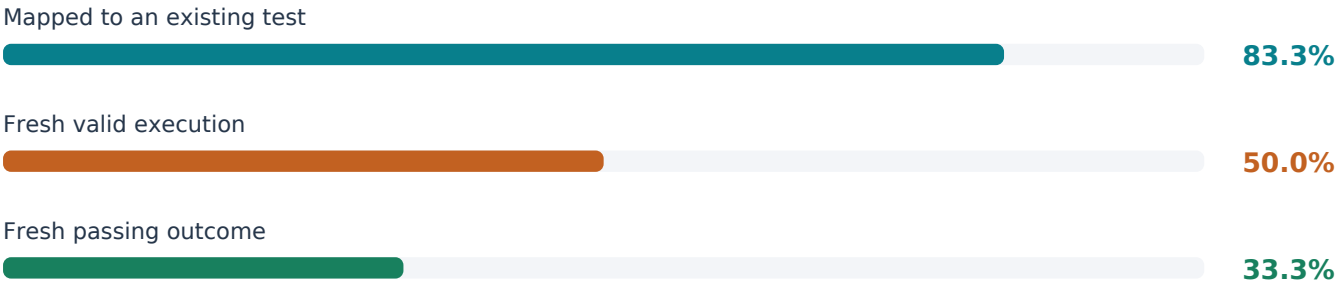
BDD already emphasizes shared examples and executable specifications. GraphWalker measures paths through a model; Hypothesis and fast-check explore action sequences; NIST documents combinatorial testing; Stryker measures sensitivity to mutations. These are useful components, not competing definitions of total correctness. [S31] [S35] [S36] [S37] [S38] [S39]

The hard product problem is defining and maintaining a useful scenario inventory - not inventing another percentage.

An honest coverage report

Illustrative checkout example. Six scenarios were approved for this change. The inventory is versioned before tests are generated. These numbers describe this defined scope only.

Scenario / expected behavior	Evidence status
Valid payment creates one order	Fresh pass on current commit.
Retry with the same idempotency key does not charge twice	Fresh product failure; duplicate charge reproduced.
Out-of-stock item does not produce a charge	Fresh pass on current commit.
Expired session cannot submit an order	Test exists; run blocked by missing login fixture.
Expired coupon is rejected with the correct total	No mapped test.
Payment timeout has a safe retry/recovery outcome	Mapped test; result is stale after relevant changes.



Mapped: 5/6. **Executed with valid current evidence:** 3/6. **Passing:** 2/6. The failing product scenario counts as exercised and remains visibly failed. Blocked, skipped, stale and untested scenarios do not count as successful execution.

Display the unknowns: show additional unapproved suggestions separately. Do not improve the percentage by quietly dropping difficult scenarios. Show inventory version, scenario count, exclusions, commit, environment, fixture identity and timestamps. Compare percentages only on the same denominator.

A 2024 study of particular LLM test generators found that their oracle design could preserve faulty behavior. A 2026 scenario-generation paper also distinguishes useful scenarios from control-flow coverage. Neither finding proves all current tools fail; both justify measuring bug detection and expected outcomes. [S65] [S66]

Foundations: seven mature options

These are potential integrations and strong free substitutes. They already support meaningful scenario execution; users still need good intent, fixtures and assertions.

Solution	What it contributes	Remaining job for your proposed tool
Playwright [S01]	Browser/API test runner plus planner, generator and healer agents.	Persist the approved scenario inventory and its coverage changes across PRs.
Cypress [S13]	E2E/component tests; cy.prompt generates commands and can self-heal.	Explain missing behaviors and verify intended outcomes independently of selector repair.
Selenium [S30]	Established cross-browser WebDriver automation.	Reduce setup and scenario-to-evidence bookkeeping around existing suites.
Cucumber [S31]	BDD collaboration and Gherkin examples that become executable checks.	Maintain links from changing requirements to current assertion evidence.
Reqnroll [S32]	Open-source .NET Given-When-Then executable specifications.	Make scenario discovery and gap review easier for existing .NET teams.
Robot Framework [S33]	Keyword-driven acceptance automation with a broad library ecosystem.	Expose stale or missing business cases without replacing existing keywords.
Karate [S34]	API/UI testing with coverage, contracts and business-rule capabilities.	Offer a very small PR-focused workflow; assess its existing coverage features closely.

The final column is an analyst assessment of the workflow opportunity. It is not a claim that a vendor lacks every related feature. Verify overlap in a hands-on comparison.

Direct AI competitors: eight to watch

These companies directly address test authoring, execution or maintenance. Several already integrate into coding agents. Do not position your product as the first AI testing CLI or MCP integration.

Solution	What it contributes	Remaining job for your proposed tool
TestSprite [S02] [S03]	Agentic frontend/backend test loop, CLI, editor integration and PR testing.	Prove less setup or better gap precision; its product already tracks use-case test status.
Momentic [S14]	AI web/mobile testing with usage-based credits.	Validate whether your local, runner-owned approach is simpler for the target repository.
Bugster [S15]	AI E2E testing and documented Claude Code-assisted test specifications.	Beat its first-run and maintenance experience in a measured comparison.
Shiplight [S16]	Coding-agent skills/MCP, readable repository YAML tests, local/hosted runs.	Differentiate on explicit scenario obligations and change coverage, not repo ownership alone.
QA Wolf [S17]	Self-service platform and managed coverage creation/maintenance.	Serve teams that want to retain their own simple workflow rather than buy managed QA.
TestMu KaneAI / Kane CLI [S04]	Real-browser/mobile verification, coding-agent integration and evidence packs.	Evidence-backed verification already exists; demonstrate a superior coverage inventory workflow.
Rainforest QA [S28]	AI test automation aimed at growing SaaS teams.	Validate a lighter, developer-operated path for teams with an existing runner.
Autify [S29]	AI software-testing platform for teams.	Focus on a narrow developer task instead of another broad platform.

The final column is an analyst assessment of the workflow opportunity. It is not a claim that a vendor lacks every related feature. Verify overlap in a hands-on comparison.

Enterprise platforms: nine incumbents

Enterprise offerings can combine broad automation, management and governance. Their existence is evidence of established demand, not evidence that a small tool can win those buyers immediately.

Solution	What it contributes	Remaining job for your proposed tool
mabl [S18]	Web/mobile/API verification; requirements-to-results traceability.	Its positioning already overlaps with independent verification; compete on a narrow task.
testRigor [S20]	Plain-English end-to-end tests across many surfaces; Claude Code MCP/skills.	Ease of authoring and agent integration are already offered.
Testsigma [S21]	Requirement-driven agents, execution and maintenance across multiple surfaces.	Do not promise exclusive ownership of requirements-to-coverage automation.
Functionize [S22]	AI-powered enterprise testing and quality verification.	Win developers with immediate local usefulness rather than broad enterprise breadth.
Katalon [S23]	Integrated end-to-end testing platform.	Integrate existing results; avoid asking teams to migrate their toolchain.
Tricentis Tosca [S24]	Enterprise model-based automation and business-process testing.	Hide modeling overhead for a constrained PR use case.
Tricentis Testim [S25]	AI-assisted UI/functional test creation and stability.	Show the semantic gap left by an otherwise stable test suite.
ACCELQ [S26]	Codeless automation across enterprise application stacks.	Start with developer-owned repositories, not broad packaged-app automation.
Keysight Eggplant [S27]	Model-based and GUI-oriented testing across application environments.	Use formal ideas while minimizing the model authoring required for the first useful result.

The final column is an analyst assessment of the workflow opportunity. It is not a claim that a vendor lacks every related feature. Verify overlap in a hands-on comparison.

Coverage and test quality: eight options

This category contains much of the actual technical foundation for meaningful coverage. Treat these tools as engines to reuse or benchmarks to beat.

Solution	What it contributes	Remaining job for your proposed tool
Hypothesis [S35]	Property-based and stateful testing in Python.	Translate a business risk into an appropriate property and understandable counterexample.
fast-check [S36]	Properties, command sequences, models and race-condition testing for JavaScript.	Help developers choose useful invariants and preconditions.
GraphWalker [S37]	Graph-based paths with edge/vertex/requirement coverage and seeded replay.	Build and maintain the model from reviewable intent with less manual work.
NIST ACTS / methods [S38]	Constrained t-way interaction coverage.	Choose valid dimensions, constraints and interaction strength for each feature.
Stryker [S39]	Mutation testing for assertion sensitivity.	Use focused mutation checks without presenting mutation score as semantic completeness.
Tricentis SeaLights [S40]	Change impact, test-gap analysis and selective execution.	Semantic business obligations plus a smaller local workflow must justify coexistence.
Qodo / Qodo Cover [S41] [S42]	Scenario expansion, quality workflows and local/CI code-coverage generation.	Measure missing behavior and actual bug detection, not only additional covered lines.
Diffblue [S43]	Enterprise automated unit-testing offering.	Keep the distinction between unit-test generation and a business scenario inventory.

The final column is an analyst assessment of the workflow opportunity. It is not a claim that a vendor lacks every related feature. Verify overlap in a hands-on comparison.

API, regression and execution: ten options

These tools cover different parts of the problem. A contract, replay or visual comparison can be powerful evidence while still leaving a business invariant untested.

Solution	What it contributes	Remaining job for your proposed tool
Postman [S44]	API test workflows, collections and execution.	Map business cases to assertions across existing requests and workflows.
Pact / PactFlow [S45]	Consumer/provider contracts and release compatibility workflow.	Cross-service compatibility is one kind of scenario protection; extend with business intent.
Schemathesis [S46]	Schema-based property tests and chained API operations.	Combine schema exploration with domain rules not expressed in the schema.
Keploy [S47]	Traffic replay and dependency mocks with code/schema coverage signals.	Add cases absent from recorded traffic; distinguish intended changes from regressions.
Meticulous [S48]	Recorded frontend interactions, code-informed test selection and visual replay.	Add explicit business outcome checks beyond recorded/visual behavior.
Applitools Eyes [S49]	AI-assisted visual checks.	Check permission, money, data and state invariants that can leave the page looking correct.
BrowserStack Percy [S50]	Visual regression and review workflow.	Connect visual evidence to the broader scenario inventory.
Chromatic [S51]	Frontend visual testing and UI review.	Retain component strengths while adding business journey evidence.
Checkly [S52]	Synthetic monitoring and reuse of Playwright checks.	Link development-time gaps and production monitors around the same scenario IDs.
BrowserStack Automate [S53]	Browser/device execution infrastructure.	Supply the scenario selection and intent layer on top of existing execution capacity.

The final column is an analyst assessment of the workflow opportunity. It is not a claim that a vendor lacks every related feature. Verify overlap in a hands-on comparison.

Management, substitutes and a closure

These eleven families complete the 53-row landscape. Test management is an integration surface; browser automation libraries and free agent workflows are substitutes. AI-agent evaluation is adjacent to testing ordinary application behavior.

Solution	What it contributes	Remaining job for your proposed tool
Qase [S54]	Test management, MCP and requirements traceability.	Automatic discovery still needs review; a lightweight inventory can feed management tools.
TestRail [S55]	Cases, runs and traceability to defined scope.	Keep requirement links tied to actual current execution, not just linked case counts.
Zephyr [S56]	Test management and Jira-centered workflows.	Provide developer-local evidence that can flow into established team records.
PractiTest [S57]	Test management and consolidated visibility.	Reduce manual bookkeeping between intent, implementation and execution.
Superpowers [S58]	Agent workflow skills with TDD and verification.	A free workflow skill is a real baseline; show measurable incremental value.
GitHub Spec Kit [S59]	Spec-driven planning and coding-agent workflows.	Import its intent artifacts rather than creating competing specifications.
OpenSpec [S60]	Repository requirements with concrete scenarios.	Map existing scenarios to assertions and fresh results.
Stagehand [S61]	AI browser automation building blocks.	A browser action engine alone is not a maintained scenario coverage model.
Midscene.js [S62]	AI UI automation and E2E testing.	Use its interaction capabilities where useful; own the intent/evidence bookkeeping.
TestMu Rook [S63]	Scenario testing for AI-agent implementations via a CLI and coding-agent skills.	Adjacent if you later support conversational/agent behavior; distinct first market.
Octomind - historical [S64]	AI E2E generation and MCP were part of its offer; company announced closure.	Do not list it as an active buying option. Its closure makes willingness-to-pay validation urgent.

The final column is an analyst assessment of the workflow opportunity. It is not a claim that a vendor lacks every related feature. Verify overlap in a hands-on comparison.

Where the proposed product must win

The opportunity is a specific combination of workflow and economics. It is not an untouched category. A missing feature in public documentation is not proof of absence.

Closest baseline	Already documented	What you must demonstrate
Native agent + Playwright [S01] [S05]	Plan scenarios, generate tests and verify a running app.	More useful accepted gaps per minute; persistent coverage history with less prompting.
TestSprite [S02] [S03]	Coding-agent entrypoints, frontend/backend test loop and visible test status.	Lower setup friction or materially better change-specific precision in your target stack.
Kane CLI [S04]	Agent-native real execution, pass/fail verdict and evidence pack.	Useful scenario inventory and explicit uncovered obligations beyond one verification run.
Shiplight [S16]	Agent skills/MCP, repository tests and local/CI execution.	Coverage analysis that remains useful with existing tests and without migration.
mabl / Testsigma [S18] [S21]	Intent/requirements-led testing and broad platform coverage.	A bounded developer task delivered with much less operational overhead.
SeaLights / management tools [S40] [S54] [S55]	Change/test gaps and requirement-result links.	Semantic gap review that developers can adopt immediately in a repository.

A defensible product claim

> **For this change, here are the important approved behaviors, the tests and assertions that check them, and the gaps that still lack current evidence.**

Avoid claiming: complete coverage of all possible scenarios, zero maintenance, guaranteed correctness, or unlimited AI usage. None follows from this architecture. The achievable promise is an inspectable, useful inventory with honest evidence states.

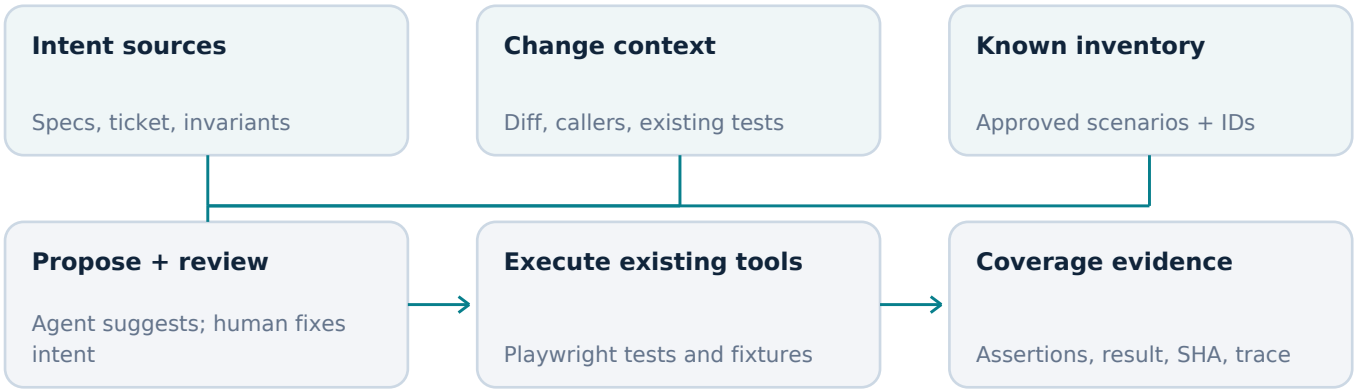
Developer pain points and product responses

Pain and evidence	Product response	What to validate
A green suite exercises code without checking the right outcome. Oracle research provides a concrete warning. [S65]	Require an intended outcome, source and assertion mapping for each accepted scenario.	Do generated tests catch a seeded domain bug, rather than encode it?
Teams do not know which cases are missing. Scenario research separates this from branch coverage. [S66]	Review candidate gaps around actor, state, event, failure, recovery and boundary.	How many proposed gaps does the developer accept as relevant?
Fixtures, auth and test data block execution. Runner best practices and Pact discuss the practical issues. [S45] [S67]	Reuse existing setup; label environment failures as blocked.	Time to first useful result on a repository with login and a database.
Brittle tests consume maintenance effort. Vendors explicitly invest in healing. [S01] [S13]	Prefer stable locators; review repairs that change expected behavior.	Maintenance minutes per accepted scenario across later PRs.
Generated suites become too large and slow. Selective execution is an established capability. [S40]	Rank gaps and select a bounded set; let users see why tests were chosen.	Useful regressions caught per CI minute; misses from selection.
Case links become mistaken for current coverage. Traceability alone is insufficient. [S55]	Record commit, environment, test version and freshness separately.	Can stale, skipped or blocked evidence incorrectly turn green?
Another account, bill or DSL creates friction. Agent integration is widespread. [S04] [S15] [S16]	Start inside the agent, with its existing login and normal test files.	Can an unfamiliar developer get a useful result without founder help?
A useful free tool may still lack a paying buyer. Octomind explicitly cited market validation. [S64]	Validate recurring team value and paid pilots early.	Do teams renew after the first impressive demo?

This is a source-informed problem map, not a survey of your target users. Interview evidence and hands-on usage must determine which pains are frequent enough to buy a product for.

A simple product, with a narrow first user

Initial buyer: a technical founder or engineering lead at a small SaaS team already using Claude Code or Codex and a working Playwright TypeScript suite. **Initial task:** inspect a feature PR before merge. This is a proposed segment, not measured market demand.



LLM judgment proposes and explains. The runner records what actually happened.
The product owns the scenario ledger and evidence links; the repository owns its tests.

Developer experience	Product behavior
Install one agent skill/plugin	Detect the repository and reuse the existing runner, setup and login state.
Ask: "What important scenarios does this change miss?"	Read the diff, available intent and existing assertions; show the five most useful candidate gaps.
Accept or correct the scenarios	Save stable scenario IDs and expected outcomes with provenance. Inferred intent stays marked as inferred.
Generate and run selected tests	Write ordinary Playwright test files. Use API-level checks when they exercise the obligation more efficiently.
Read a concise report	Show verified passes, reproduced failures, blocked execution, missing tests and stale results separately.

Working concept name: "Scenario coverage for your PR." Use a final brand only after checking repository, package and domain availability. The product should be understandable before the user learns its name.

Subscription integration: feasible, with limits

Integration	Recommended first approach	Boundary
Claude Code	A repository skill/plugin that calls your deterministic CLI. Optional adapter can use claude -p. [S05] [S12]	User keeps the official CLI and account. Current Anthropic guidance supports subscription usage for these paths. [S10]
Codex	A repository skill in .agents/skills; optional codex exec adapter. [S06] [S11]	ChatGPT subscription authentication and separately billed API credentials are different routes. [S07]
Local and CI test replay	Run saved ordinary tests and import structured runner results.	Replay need not invoke an LLM. It still consumes the user's local or CI compute.
Unattended hosted generation	Later: customer-approved API/provider access or an expressly supported account integration.	Do not base pooled server inference or resale on consumer subscription credentials.

What the marketing can honestly say

Works inside Claude Code and Codex with your existing agent login. No additional model API key for the local agent-driven workflow. Normal provider limits still apply.

Privacy: local orchestration can avoid sending source code to your service by default. The selected coding agent and model provider still process the context they receive. A local harness does not make cloud-model inference stay on the laptop.

Operational limits: rate limits, model access, context length, CLI updates, app startup and permissions affect runs. Detect unavailable agents and show clear next steps. Do not start nested agents from a skill when the existing session can do the work.

Current-source caveat: Anthropic's October 7, 2026 help update allows current subscription usage and notes plan API credits. This is evidence for present feasibility, not a permanent third-party billing guarantee. Keep adapters replaceable. [S10]

MVP scope and six-week implementation plan

Estimate for one experienced founder working full time on a constrained alpha. A general solution across languages, runners and environments would take substantially longer.

Wk	Deliverable	Exit evidence
1	Interview 10-15 developers; manually analyze 10 recent PRs. Define the scenario record and a single runner adapter.	Developers agree the top gaps are real; normal test setup can be reused.
2	Ship a local CLI and shared skill logic for Claude Code and Codex. Import Playwright results; support a tiny fixture repo.	Unassisted first report in under 5 minutes on the supported fixture.
3	Candidate generation, human review, stable IDs and assertion links. Generate ordinary tests for approved cases.	At least three useful, distinct gaps on representative PRs; no fabricated pass evidence.
4	Freshness rules, failure classification, HTML/Markdown output and optional PR summary.	Stale, skipped and blocked runs cannot satisfy the passing metric.
5	Compare against native-agent baselines on held-out regressions. Onboard five design partners.	Measured gap precision, caught bugs, setup time and support effort.
6	Harden the supported path, publish benchmark artifacts and launch the repository. Ask teams to pay for shared workflow value.	Repeated weekly usage and concrete paid-pilot commitments.

Keep the first release small

Include: TypeScript + Playwright, a running local/staging app, existing fixtures, five ranked suggestions, native test generation, a versioned scenario ledger, structured results and a local report. A Playwright API test is acceptable; not every scenario requires a browser.

Defer: a desktop office UI, a new browser engine, all languages/runners, mobile, Jira setup, production traffic collection, autonomous production testing, mandatory accounts, and full self-healing. Add one adapter only when demand justifies it.

Suggested implementation: TypeScript CLI; Git diff and runner adapters; schema-validated scenario metadata; JSON result ledger; static HTML report. Use the agent for semantic judgment and normal tooling for execution. Do not add a vector database or a multi-agent platform before the first user needs it.

Design the coverage model before the badge

Record	Minimum fields
Scenario obligation	Stable ID; actor; preconditions; event/action; expected outcome; intent source; risk priority; accepted/inferred state; inventory version.
Test mapping	Runner test ID; test file/version; assertion location and expected value/property; scope and known exclusions; reviewer approval state.
Execution evidence	Test invocation; exit/result; scenario ID; commit/tree state; environment and fixture hash; timestamp; trace/log artifact; freshness status.
Gap or uncertainty	Missing assertion, absent test, blocked setup, invalid/stale evidence, ambiguous intent or unsupported dependency path.

A transparent set of metrics

Inventory execution coverage = scenarios with valid fresh execution / approved in-scope scenarios. Count a reproduced product failure as exercised, while retaining the failed verdict.

Passing scenario share = scenarios with valid fresh passing outcomes / approved in-scope scenarios. Report the numerator and denominator as well as the percentage.

Optional priority-weighted share = sum of agreed weights for fresh passing scenarios / sum of weights for the fixed approved inventory. These weights are prioritization choices, not calibrated probabilities of business loss.

Additional signals: approved new gaps, suggestion acceptance, stale/blocked counts, test sensitivity to targeted mutations, and regressions reproduced. Do not average unrelated measures into an unexplained confidence score.

Trust rules that matter

An agent narrative is not execution evidence. Import runner output and artifacts. A passing mapped test is not automatically a correct oracle: review the asserted business outcome and its source. Static dependency links are incomplete for reflection, feature flags and dynamic calls; mark uncertainty rather than claiming perfect impact analysis.

Freeze the comparison inventory for a PR. If discovery adds approved scenarios, version the denominator and explain the change. A new test can raise executed coverage without raising passing outcomes when it correctly reveals a real bug.

Public pricing shows several buying models

Prices below are the public snapshot checked for this study. Different credit definitions make direct dollar-per-test comparisons misleading. Browser time, AI steps, retries, support and contract scope all matter.

Solution	Observed public pricing	Interpretation
Playwright + existing agent	Framework is free; agent plan and compute are separate. [S01]	Your strongest entry-price competitor. Show value beyond a good prompt.
TestSprite	Free 150 credits/month. Starter $19/month; Standard $39/month; Pro $69/month. [S03]	A low-priced coding-agent competitor already exists. Compare actual workload consumption.
Momentic	Official indexed pricing: Free 2,000 credits/month; paid from $125/month plus usage. [S14]	Usage pricing; direct pricing-page extraction failed. Treat detailed quotas as unverified here.
QA Wolf	Platform: $0.01/AI credit + $0.15/runner minute. Managed coverage: quote-led by tests under management. [S17]	Sells both self-service infrastructure and relief from maintenance work.
mabl	Quote/custom-plan route; cloud-run credits; local tests included. [S19]	Platform economics differ from a small local utility.
Munder Difflin	Repository advertises a $150/year Pro launch offer, with regional adjustment. [S09]	Free core plus optional paid convenience. Public offer does not establish revenue or conversion.

A pricing principle for your tool

Let developers discover and reproduce gaps for free. Charge teams for shared history, review workflow and organizational policy.

Do not sell "more generated tests" as the primary value unit. More tests can mean more noise and maintenance. Tie the paid value to repeated release decisions and collaboration across repositories.

A business model worth testing

Proposed experiments, not market-validated prices. Start by selling to five small teams; keep the useful local workflow complete so GitHub adoption does not depend on creating an account.

Offer	Proposed price	What a customer gets
Community	Free, open-source core	Local CLI and agent skills, complete local scenario ledger, deterministic test replay, local reports and CI integration. Core evidence rules remain free.
Personal convenience	Test $99/year	Optional polished desktop/report workspace and private sync. Build only if repeated individual usage demonstrates demand.
Team workspace	Test $99/month for up to 5 active repositories	Shared scenario ownership, PR review history, baselines, policy controls and a bounded evidence-retention allowance. Viewers need not be paid seats.
Larger teams	Later: explicit quote	Private deployment, admin integrations, service commitments and larger retention. Sell after the small-team workflow works.

Economics to calculate in the pilot

Illustrative $99/month workspace: $8 storage/egress + $7 application infrastructure + $3 payment cost + $10 expected support = $28 variable monthly cost, leaving $71 contribution (about 72%). These are assumptions, not observed costs; they exclude founder salary, acquisition, taxes, refunds and R&D. Heavy video retention or support can erase the margin.

This model assumes model work and runner compute stay with the customer. If you host execution or inference, meter those separately and remeasure costs. Do not imply unlimited cloud runs inside the same $99 plan.

A bottom-up opportunity, not a fabricated TAM

100 paying workspaces at $99/month = $9,900 MRR. 1,000 = $99,000 MRR. These are arithmetic scenarios, not forecasts. Track the number of reachable teams with the target stack, observed activation, conversion and retention before estimating a market size.

Commercial hypothesis: teams pay to maintain a shared, trustworthy coverage history that helps a reviewer decide what still needs testing. A one-time scenario brainstorm is more likely to remain a free feature.

What to learn from Munder Difflin

Munder Difflin wraps existing terminal agents in a local, open-source experience, with a memorable visual interface and optional paid additions. Its repository is MIT licensed and advertises a Pro annual launch offer. This is evidence of packaging and monetization choices, not verified product revenue. [S08] [S09]

Pattern to borrow	Translate it into scenario testing
Use what the developer already pays for	Run within the existing Claude Code or Codex session; offer adapters instead of another mandatory model subscription.
Make the work visible	A compact scenario map highlights tested, failing, blocked and missing outcomes. Clicking a cell opens its test and evidence.
Provide an immediately useful free core	One repository command and a first useful gap report; no account required for local use.
Make the demo understandable without explanation	A green existing suite misses a duplicate charge. Your tool reproduces it and saves a normal regression test.
Offer paid convenience after utility is established	Shared evidence history and team workflows; optional personal interface after demand appears.
Use a recognizable visual identity	Create an original risk-map visual treatment. Animation should show testing progress rather than decorate an otherwise unclear product.

What will not create a durable advantage

A prompt pack, an MCP server, a colorful dashboard or subscription reuse can all be copied. Potential durable assets are corrected intent-to-test mappings, reliable fixture integrations, low-noise change analysis, a benchmark and useful team history. These become advantages only if users repeatedly choose them.

Copy the distribution logic: obvious utility, low setup and a shareable result. Earn the business through repeat use and a paying team buyer.

A GitHub launch built around a real result

Virality cannot be promised. Design the repository so a developer can independently reproduce the result and share something useful. The best launch asset is a short, credible demonstration of a missed bug.

Asset	Concrete execution
30-45 second demo	Start with an existing suite that passes. Inspect a small change; show the missing retry scenario; run it; reveal a seeded duplicate-charge bug; save the regression test.
Second and third demos	Tenant authorization: access to another tenant must fail. CSV import: duplicate uploads must be idempotent and invalid rows must not corrupt valid data.
README opening	One sentence about missed scenarios, a short clip, one verified install path, supported stack, a sample report and a "what this measures" explanation.
Reproducible fixture	Ship a tiny permissively licensed demo app with labeled intentional bugs, existing tests and a one-command reset. Keep seeded failures clearly labeled.
Shareable report	A redacted PR image or local report shows "12 approved scenarios: 7 pass, 1 fail, 2 blocked, 2 untested". Never invent a universal completeness badge.
Community surface	Good first issues for one adapter at a time, scenario examples, clear contribution instructions, a changelog and published baseline results.
Distribution sequence	Design partners first; then a technical launch post, Show HN, relevant developer communities and a small set of authentic demonstration videos. Show benchmark artifacts, not only claims.

A product loop that can support sharing

A PR exposes a useful gap; a developer saves a normal regression test; the review shows the result; a teammate installs the tool on another repository. Public reports and optional badges can make that result visible. Track these transitions instead of assuming every star becomes a user.

North-star candidate: weekly repositories where a developer accepts a useful scenario and produces a reproducible result. Stars, impressions and package downloads are distribution signals, not customer outcomes.

Validate against the free baseline

Before building a broad platform, run a controlled comparison on held-out feature changes. The central question is whether your workflow adds useful coverage beyond the same coding agent with an appropriate prompt and ordinary runner tools.

Experiment	Design
Benchmark corpus	At least 20 held-out changes across 3 small repositories, with 30 documented seeded regressions. Include auth, retries/idempotency, state boundaries, import/data validation and recovery.
Ground truth	Have maintainers or a separate reviewer approve the intended behavior before the evaluated run. Record fixtures, allowed evidence, bug triggers and accepted scenario families.
Baselines	Native agent + existing tests; agent + Playwright planning workflow; your tool on the same agent. Optionally compare a paid platform on an agreed workload.
Fairness	Use the same repository state, models, environment, time/usage budgets and prompts where applicable. Repeat at least 3 times to expose variability. Separate discovery and test execution costs.
Primary outcomes	True domain regressions reproduced; precision of top-five suggestions; bugs missed; time to a useful result; runnable-test rate; duplicate/noise rate; reviewer minutes.
Oracle audit	Check that a generated assertion reflects the approved requirement. Exclude cases where a test merely copies the implementation's erroneous output.
Publishable artifacts	Commit/fixture IDs, scenario inventory versions, commands, generated tests, results and failure artifacts. Report limitations and small-sample uncertainty.

Five design partners, then a decision

Recruit small SaaS teams already using agents and Playwright. Observe their next two real PRs. Ask for a concrete paid trial once they have seen value; do not substitute "would you pay?" enthusiasm for a purchase. Review which accepted scenarios remain useful a month later.

Prototype targets, to be revised after the first cohort: first useful report within 5 minutes on supported repositories; at least 70% of top-five suggestions judged relevant; under 10% environment failures wrongly called product bugs; clear incremental regression detection over the native baseline. These are decision thresholds, not industry benchmarks.

Go, narrow, or stop: the first 90 days

Period	Focus	Evidence to continue
Days 1-14	Interviews, manual PR analysis, one-stack prototype and baseline comparison.	Real missing cases that the existing workflow did not surface efficiently.
Days 15-45	Five design partners, repeated use, fixture reliability and initial paid offers.	At least 3 of 5 teams still use it weekly after 4 weeks; at least 2 make paid commitments.
Days 46-90	Open-source launch, small-team workspace and retention measurement.	Users return after the demo; onboarding and support costs are compatible with the tested price.

Main risks and practical decisions

Risk	Decision rule
The tool is equivalent to a skill and a good prompt.	If it cannot beat the free native baseline on useful results or effort, keep it a small open-source utility or narrow the task.
Too many speculative scenarios create review work.	Reduce scope and show only the best five. Measure developer acceptance and missed bugs together.
Environment support dominates engineering.	Support one repeatable app/runner path; charge for truly recurring shared value rather than unlimited setup assistance.
Native vendors add the same workflow.	Integrate their execution improvements. Recheck whether inventory/history and cross-tool evidence still attract users.
A coverage score creates false confidence.	Keep unknowns and stale/blocked states visible. Never describe model coverage as exhaustive application coverage.
Launch attention does not convert to repeat use.	Prioritize cohort retention, accepted scenarios and paying workspaces over star-count targets.

Recommendation: build a small open-source experiment now. Earn a larger product by proving recurring, incremental value on real PRs.

The strongest initial story is simple: "Your tests passed. Here are the important scenarios they missed." Back it with a reviewable inventory and real execution artifacts. A reliable small tool can be a good business; becoming the most viral GitHub repository is an aspiration, not an outcome the study can forecast.

Sources and verification notes

All sources accessed or checked on 15 October 2026. Product descriptions are primary vendor/repository documentation, not independently verified performance. Research papers are primary research. Indexed-only or limited extraction is marked.

[S01] Playwright Test Agents playwright.dev / view page
Planner, generator, healer and agent-loop setup.

[S02] TestSprite product testsprite.com / view page
Agentic testing, test statuses, CLI and failure artifacts.

[S03] TestSprite pricing testsprite.com / view page
Current public plans and credit allowances.

[S04] TestMu AI Kane CLI testmuai.com / view page
Agent integration and evidence-backed web/mobile verification.

[S05] Claude Code skills code.claude.com / view page
Local skills, bundled /run and /verify workflows.

[S06] Codex: Build skills learn.chatgpt.com / view page
Repository skill discovery and plugin distribution.

[S07] Codex authentication learn.chatgpt.com / view page
Subscription sign-in and API authentication are distinct.

[S08] Munder Difflin website munderdiffl.in / view page
Free local harness, existing subscriptions and paid product additions.

[S09] Munder Difflin repository github.com / view page
MIT core, architecture and displayed Pro annual launch offer.

[S10] Claude plan and Agent SDK usage support.claude.com / view page
October 7 update: current subscription usage support; billing can change.

[S11] Codex non-interactive mode learn.chatgpt.com / view page
codex exec, automation and sandbox controls.

[S12] Claude Code programmatic execution code.claude.com / view page
claude -p and structured execution options.

[S13] Cypress AI test generation docs.cypress.io / view page
Natural-language test steps and self-healing.

[S14] Momentic product and pricing momentic.ai / view page
Official indexed page: Free 2,000 credits/month; paid from $125/month plus usage. Direct fetch rejected text/markdown.

[S15] Bugster Claude Code integration docs.bugster.dev / view page
Coding-agent test-generation integration.

[S16] Shiplight product shiplight.ai / view page
Agent skills/MCP, repository YAML tests, local and hosted execution.

[S17] QA Wolf pricing qawolf.com / view page
Self-service credit/minute rates and managed coverage offering.

[S18] mabl product mabl.com / view page
Verification harness and requirement-to-result traceability.

Sources and verification notes

[S19] mabl pricing mabl.com / view page
Custom plans, cloud run credits and included local tests.

[S20] testRigor product testrigor.com / view page
Plain-English tests, broad surfaces, Claude Code MCP/skills.

[S21] Testsigma product testsigma.com / view page
Requirement-driven agents and web/mobile/API/Salesforce testing.

[S22] Functionize product functionize.com / view page
Enterprise automation and verification positioning.

[S23] Katalon product katalon.com / view page
Integrated testing platform.

[S24] Tricentis Tosca tricentis.com / view page
Enterprise model-based test automation.

[S25] Tricentis Testim testim.io / view page
AI-assisted UI and functional testing.

[S26] ACCELQ accelq.com / view page
Codeless automation across enterprise applications.

[S27] Keysight Eggplant Test keysight.com / view page
Model-based and GUI-oriented automation.

[S28] Rainforest QA rainforestqa.com / view page
AI test automation for SaaS teams.

[S29] Autify autify.com / view page
AI software-testing platform.

[S30] Selenium WebDriver selenium.dev / view page
Browser automation foundation.

[S31] Cucumber: BDD cucumber.io / view page
Example-led discovery, formulation and automation.

[S32] Reqnroll docs.reqnroll.net / view page
Executable Given-When-Then specifications for .NET.

[S33] Robot Framework robotframework.org / view page
Keyword-based acceptance automation ecosystem.

[S34] Karate Labs karatelabs.io / view page
API/UI testing, coverage and business-rule testing.

[S35] Hypothesis stateful tests hypothesis.readthedocs.io / view page
Generated action sequences and state-machine testing.

[S36] fast-check model-based testing fast-check.dev / view page
Command sequences, preconditions and independent models.

Sources and verification notes

[S37] GraphWalker graphwalker.github.io / view page
Graph traversal, edge/requirement coverage and replay seeds.

[S38] NIST SP 800-142 csrc.nist.gov / view page
Combinatorial interaction testing and its scope.

[S39] Stryker mutation testing stryker-mutator.io / view page
Mutation testing as a test-sensitivity signal.

[S40] Tricentis SeaLights tricentis.com / view page
Test-gap and change-impact analysis.

[S41] Qodo testing qodo.ai / view page
Scenario expansion and agentic quality workflows.

[S42] Qodo Cover repository github.com / view page
Local/CI test generation aimed at code coverage; AGPL-3.0.

[S43] Diffblue diffblue.com / view page
Official enterprise unit-testing product headline; limited page extraction.

[S44] Postman API testing postman.com / view page
API test workflows and execution.

[S45] Pact pact.io / view page
Consumer/provider contract testing and broker workflow.

[S46] Schemathesis schemathesis.readthedocs.io / view page
Schema-driven property tests and chained API operations.

[S47] Keploy keploy.io / view page
Traffic recording/replay, dependency mocks and schema coverage.

[S48] Meticulous: How it works meticulous.ai / view page
Recorded interactions, code-informed flow selection and visual replay.

[S49] Applitools Eyes applitools.com / view page
AI-assisted visual comparison.

[S50] BrowserStack Percy browserstack.com / view page
Visual regression testing and review.

[S51] Chromatic chromatic.com / view page
Frontend visual testing and review.

[S52] Checkly checklyhq.com / view page
Synthetic monitoring, Playwright reuse and agent interfaces.

[S53] BrowserStack Automate browserstack.com / view page
Browser/device execution infrastructure.

[S54] Qase qase.io / view page
Test management, MCP and requirements traceability.

Sources and verification notes

[S55] TestRail: coverage and traceability testrail.com / view page
Relationship between defined scope, tests, execution and requirements.

[S56] Zephyr smartbear.com / view page
Test management and Jira-oriented workflows.

[S57] PractiTest practitest.com / view page
Test management and visibility.

[S58] Superpowers github.com / view page
Agent workflow skills including TDD and verification.

[S59] GitHub Spec Kit github.com / view page
Spec-driven planning and agent execution workflows.

[S60] OpenSpec github.com / view page
Repository requirements and concrete scenarios.

[S61] Stagehand docs.stagehand.dev / view page
AI browser automation building blocks.

[S62] Midscene.js midscenejs.com / view page
AI UI automation and E2E testing.

[S63] Rook with Claude Code testmuai.com / view page
Adjacent category: testing AI-agent implementations.

[S64] Octomind closure announcement linkedin.com / view page
First-person post embeds company closure notice; historical comparator.

[S65] Test-generator oracle study arxiv.org / view page
Mathews and Nagappan, 2024: generators can encode existing bugs in expected outcomes.

[S66] TestGeneralizer research arxiv.org / view page
Qi et al., 2026: scenario coverage is not reducible to branch coverage.

[S67] Playwright best practices playwright.dev / view page
Isolation, user-visible behavior, test data and resilient assertions.

Method: direct vendor/repository pages were preferred. Search results were used to discover current changes and reconcile stale listings. The Octomind closure notice overrules old product/pricing pages. Momentic pricing was checked in the indexed official page because direct retrieval rejected its content type. No private contracts, customer interviews, usage telemetry or product performance tests were available for this study.

What remains to verify: actual first-run setup, bug detection and false positives; paid-team retention; code/test export details; every provider's supported account flow for any later hosted product; full commercial pricing for quote-led vendors.